

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 08 · LECTURE NOTES

Voronoi Diagrams

Partitioning space by who is nearest.

MODULE OUTCOMES

Define the Voronoi diagram and state its structural properties.

Explain why it has linear complexity.

Implement Fortune's sweep and explain the beach line.

Apply the lifting map and connect it to convex hulls.

Name the variants and what they are used for.

1 Structure

THE DEFINITION AND THREE CONSEQUENCES

Given n sites in the plane, the Voronoi cell of a site is the set of all points strictly closer to it than to any other site. The cells partition the plane; the boundaries are the points equidistant from two or more sites.

Each cell is convex, because it is the intersection of $n - 1$ half-planes — one for each other site, bounded by the perpendicular bisector of the two. Convexity is immediate and it is what makes the cells useful downstream.

Every cell is nonempty (a site is in its own cell), and **the unbounded cells are exactly those belonging to convex hull vertices** — a site has an unbounded cell if and only if it is on the hull (Module 03), which is a pleasing link between the two structures.

And the whole diagram has only $O(n)$ vertices and edges, despite each cell being defined as an intersection of $O(n)$ half-planes and the total count of half-planes being quadratic. **That is the non-obvious structural fact**, and it is what makes the diagram computable in $O(n \log n)$.

$$V - E + F = 2 \quad F = n + 1 \quad \deg(v) \geq 3 \Rightarrow V \leq 2n - 5, \quad E \leq 3n - 6$$

The diagram is a planar straight-line graph, so Euler's formula applies with $F = n + 1$ — the n cells plus the unbounded outer face. Every Voronoi vertex has degree at least three (it is equidistant from at least three sites), and summing degrees gives $2E \geq 3V$. Combining the two yields the linear bounds. **Planarity is doing all the work here**, and the same argument bounds the size of any planar subdivision — which is worth remembering, because it is the reason planar structures are tractable and their three-dimensional analogues frequently are not.

A VORONOI VERTEX IS EQUIDISTANT FROM THREE SITES

A vertex of the diagram is a point where three or more cells meet, so it is equidistant from three sites — which makes it exactly the circumcentre of the triangle those sites form.

And the circle through those three sites contains no other site. If it did, that site would be closer to the vertex than the three are, contradicting the vertex being on all three cell boundaries. **The empty circumcircle property is the definition in disguise.**

So the predicate needed is incircle (Module 02 §2): given three sites and a fourth, is the fourth inside their circumcircle? Every Voronoi and Delaunay algorithm is built on this one test.

Four cocircular sites produce a degree-4 vertex instead of two degree-3 vertices joined by a zero-length edge. That is the degeneracy, it is why the incircle predicate must be exact, and it is exactly the configuration that makes the dual Delaunay triangulation non-unique (Module 09 §1).

2 Fortune's algorithm

WHY A NAIVE SWEEP FAILS: a Voronoi vertex can be discovered BEFORE the sweep line reaches the site that created it, so the output is not monotone in the sweep direction.

THE FIX -- the BEACH LINE: each site already passed contributes a PARABOLIC arc (points equidistant from that site and the sweep line). The beach line is the lower envelope of those parabolas, and everything above it IS already determined.

STATUS: the sequence of arcs, in x order

EVENTS: SITE -- sweep reaches a new site: split the arc above it
CIRCLE -- an arc shrinks to zero: its neighbours become adjacent, and a VORONOI VERTEX is emitted there

$O(n \log n)$, and optimal.

The beach line is the invention, and it is worth seeing what it buys: it makes the determined region monotone in the sweep direction, which is precisely the property a plane sweep (Module 04) requires and which the Voronoi diagram does not naturally have. Fortune's contribution was recognising that sweeping a *curve* rather than a line restores it.

FALSE CIRCLE EVENTS ARE NORMAL, NOT AN ERROR PATH

A circle event is queued whenever three consecutive arcs on the beach line could converge to a point. It records where a Voronoi vertex would appear if nothing intervenes.

But a later site event may split one of those three arcs first, destroying the configuration. The queued event is then invalid and must not produce a vertex.

The standard handling is lazy deletion: each arc holds a pointer to its pending circle event, and splitting or removing the arc marks that event invalid. Nothing is removed from the priority queue, which would cost a search.

Invalid events are popped and discarded. This happens constantly — it is not an exceptional case, and an implementation that logs a warning or asserts on it will drown. This is the single most common source of confusion when implementing Fortune's algorithm, because the published descriptions tend to mention it in passing.

3 The lifting map

$$(x, y) \rightarrow (x, y, x^2 + y^2)$$

Lift each site from the plane onto the paraboloid $z = x^2 + y^2$. **The lower convex hull of the lifted points, projected back down, is exactly the Delaunay triangulation of the original sites** — and the Voronoi diagram is its dual. One three-dimensional convex hull therefore yields both diagrams. **This is Module 03's parabola reduction, one dimension up**, and it is the most satisfying result in the course.

WHY THE LIFTING MAP MATTERS PRACTICALLY, NOT JUST AESTHETICALLY

Delaunay and Voronoi require no new algorithm. A working 3D convex hull implementation (Module 03 §3) computes both, which is a substantial saving in code and in the number of things that can be wrong.

It explains where the incircle predicate comes from. 'Is d inside the circle through a, b, c ?' becomes 'is lifted d below the plane through lifted a, b, c ?' — which is `orient3d` on the lifted points. **The 4×4 incircle determinant of Module 02 is the 3×3 orientation determinant after lifting**, which is why its rows contained $x^2 + y^2$.

And it generalises to every dimension. A Voronoi diagram in d dimensions is a convex hull in $d + 1$, so one piece of code handles all of them — subject to the output-size blowup of Module 03's last table.

It is also the reason qhull computes Delaunay triangulations despite being a convex hull program: it is simply applying this map. Knowing that explains the otherwise-puzzling shape of its interface.

4 Variants

Variant	What changes	What it is used for
Power (Laguerre) diagram	Sites carry weights; distance becomes the power distance, so bisectors are still straight but shift toward the lighter site.	Spheres of differing radii — molecular surfaces, packed particle systems, and weighted fracture.
Centroidal Voronoi tessellation (CVT)	Each site is required to be the centroid of its own cell. Reached by Lloyd's algorithm: compute the diagram, move each site to its centroid, repeat.	Remeshing, stippling, and blue-noise sampling. The cells become near-hexagonal and highly uniform. The variant that appears most often in graphics , and the link to CSCE 647's sampling material.
Generalised Voronoi	Sites are line segments or polygons rather than points, so bisectors become parabolic arcs.	The medial axis — skeletons, toolpath generation, and navmesh spine extraction (Module 11).
Order-k Voronoi	Cells defined by the k nearest sites rather than the single nearest.	k -nearest-neighbour queries answered by point location (Module 10).
Farthest-point Voronoi	Cells defined by the farthest site. Only hull vertices get nonempty cells.	The smallest enclosing circle , and other farthest-point problems.

Variant	What changes	What it is used for
Manhattan or L^∞ metrics	A different distance function; bisectors become polylines.	Grid-based worlds and VLSI routing, where the metric genuinely is not Euclidean.

WHERE THIS SHOWS UP IN AN ENGINE

Fracture and destruction. A Voronoi partition of a solid produces convincing shards, and because every cell is convex (§1) the pieces are immediately usable as collision proxies with no decomposition step (Module 03 §3, Module 12).

Procedural texture and terrain. Worley noise is a Voronoi distance field evaluated per pixel, and it is the standard source of cells, cracks, scales, stones, and crystalline structure.

Blue-noise sampling and stippling via Lloyd relaxation. The resulting point sets have far better spectral properties than uniform random ones, which matters for every sampling job in CSCE 647 Module 06 — from anti-aliasing to importance sampling to dithering.

Territory and influence maps, NPC spacing, and the medial axis for skeleton extraction and navigation-mesh spine generation (Module 11). **Voronoi is one of the few structures in this course that is as useful for gameplay logic as for geometry.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
de Berg et al. — Computational Geometry, chapter 7 (Voronoi diagrams)	Fortune's algorithm with the beach line and the circle-event handling of §2, done in full.
Mount — CMSC 754, Voronoi and lifting lectures (free PDF)	The lifting map of §3 , derived rather than asserted.
Aurenhammer — Voronoi Diagrams: A Survey of a Fundamental Geometric Data Structure	The variants of §4, comprehensively. Still the best single survey.
Du, Faber & Gunzburger — Centroidal Voronoi Tessellations (free)	Lloyd's algorithm and CVT, with the convergence theory.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Compute a Voronoi diagram by brute-force half-plane intersection, one cell at a time, and verify the linear total size.
- 2 **Verify the Euler bound empirically** for many random site sets.
- 3 Confirm that the unbounded cells are exactly the convex hull vertices.
- 4 Implement Fortune's algorithm. **Count how many circle events turn out to be false** and report the fraction.
- 5 Run Fortune's on cocircular sites and on collinear sites. Report what happens.
- 6 **Implement the lifting map** and compute Delaunay via your 3D convex hull from Module 03. Compare against Fortune's output.
- 7 Verify that incircle equals orient3d on lifted points.
- 8 **Implement Lloyd's algorithm** and watch a random point set become blue noise. Plot the power spectrum before and after.
- 9 Use a Voronoi partition to fracture a convex polygon and render the shards.
- 10 Implement Worley noise from your diagram and texture a surface with it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define the Voronoi diagram and give three structural consequences.
- 2 Why is the diagram linear in size despite quadratically many half-planes?
- 3 What is a Voronoi vertex geometrically, and which predicate decides it?
- 4 Why does a straight sweep line fail, and what does the beach line fix?
- 5 What is a false circle event and how is it handled?
- 6 State the lifting map and give three practical consequences.
- 7 Name six Voronoi variants and one use of each.
- 8 Give four engine uses of Voronoi diagrams.